

Galois cipher explained

Sonya Polyanskaya

21 сентября 2026 г.

1 Введение

Представим, что наша цель - уметь шифровать произвольный файл и расшифровывать его обратно. Любой файл представлен как последовательность байтов, которую надо преобразовать в другую последовательность, чтобы:

- 1) само преобразование было простым и быстрым
- 2) без знания секрета восстановить исходное было невозможно
- 3) со знанием секрета восстановление было бы мгновенным

Классический способ добиться этого - потоковое шифрование. Идея проста, представим что у нас есть гамма - длинная последовательность байтов, которая выглядит случайной, но на самом деле детерминирована и полностью определена нашим "секретом". К тому же, она должна быть непредсказуемой и не должна повторяться слишком быстро, т.е. должна иметь периодичность. Чтобы получить такую гамму, будем использовать регистр сдвига с линейной обратной связью в варианте Галуа над определенным конечным полем. Далее по разделам следует описание интерфейса нашей программы и разъяснение теории, которая лежит в основе.

2 Поле

Начнем с нижнего уровня. Наше поле это $GF(2^n)$. Почему? Мы искали поле, в котором элементы будут битовыми строками длины n и стремимся, чтобы один элемент поля был ровно одним состоянием регистра. В выбранном поле ровно 2^n элементов и каждый записывается как n битов. Сложение в нем XOR (обратная операция, удобная для шифра и наложения и снятия гаммы), а умножение задается неприводимым многочленом f степени n над $GF(2)$, редукция по нему позволяет обновлять состояние регистра.

Теперь рассмотрим правила, по которым работает наше поле. Сложение задается XOR. Умножение похоже на обычное умножение многочленов, но с редукцией по модулю f . Для шифра мы будем использовать умножение на x , которое обновляет состояние регистра на каждом шаге. Это умножение

задает циклический обход всех ненулевых элементов поля, если f выбран правильно, то последовательность

$$1, x, x, x, x, \dots, x^{2^n-2}$$

пробежит все $2^n - 1$ ненулевых элементов поля, прежде чем вернуться к 1. Это нам нужно, чтобы получить длинный, неповторяющийся цикл.

Теперь про редукцию. Когда мы умножаем элемент поля на x , старшая степень многочлена увеличивается на единицу. Если элемент имел степень $n - 1$ (то есть был на самой границе допустимого), то после умножения на x получится x^n — степень ровно n , а это уже выше того, что помещается в наш регистр из n битов. Регистр физически не может хранить $n + 1$ бит: у него ровно n ячеек. Значит, от x^n надо как-то избавиться, но так, чтобы не потерять информацию, иначе последовательность вырождается и цикл оборвётся. Здесь и вступает в игру редукция. Многочлен $f(x)$ по определению равен нулю в нашем поле. Запишем его как $x^n + c_{n-1}x^{n-1} + \dots + c_1x + c_0 = 0$. Перенесём x^n в правую часть: $x^n = c_{n-1}x^{n-1} + \dots + c_1x + c_0$. Это правило замены: всякий раз, когда появляется x^n , мы заменяем его на комбинацию младших степеней, заданную f .

Что это даёт на практике? После умножения на x и сдвига влево старший бит вылетает за пределы n разрядов. Редукция «заворачивает» его обратно — но не в одну младшую позицию, а в несколько сразу, определяемых порождающим. Это гораздо сильнее перемешивает информацию, чем простой циклический сдвиг, и именно поэтому последовательность состояний не закикливается раньше времени. Если бы мы просто отбрасывали вылетевший бит, состояние постепенно теряло бы информацию и в какой-то момент обратилось бы в ноль. Если бы заворачивали его в одну позицию, получился бы обычный циклический сдвиг с коротким периодом. Редукция даёт третье: сохраняет всю информацию внутри регистра, но распределяет её так, что состояние обходит все $2^n - 1$ ненулевых элементов поля по одному разу. Именно поэтому f должен быть неприводимым: если бы он разлагался на множители, редукция не задавала бы поле, и x не был бы генератором всего цикла.

Неприводимость гарантирует, что замена x^n на $c_{n-1}x^{n-1} + \dots + c_0$ — корректная операция поля, и что при правильном выборе f мы получаем максимально длинный цикл $2^n - 1$. Без редукции регистр либо вырождается, либо закикливается слишком быстро. С редукцией он превращается в надёжный источник гаммы, из которого мы дальше будем извлекать биты.

Таким образом, наше поле — инструмент для генерации длинной последовательности без повторов.

3 Регистр

Регистр — это просто память фиксированного размера. Внутри регистра лежит число. Состояние — это то, что сейчас лежит в регистре. Каждый шаг состояние меняется. Шаг — это переход от одного состояния к другому. У нас шаг — это умножение состояния на x .

Зачем нужны шаги? Затем, что мы хотим получить последовательность состояний: $S_0 \rightarrow S_1 \rightarrow \dots$. Каждый шаг даёт новое состояние, из которого мы можем извлечь новый бит гаммы. Так из одного начального состояния мы получаем длинную последовательность битов. Мы хотим, чтобы новое состояние было «перемешанной» версией старого. Нам нужно сложное перемешивание, при котором каждое состояние сильно зависит от предыдущего и не повторяется долго.

Используем умножение на x . Каждый бит переезжает на одну позицию в сторону старших, а на месте младшего появляется 0. Информация не потерялась, но перераспределилась. Но есть проблема: старший бит вылетает. При сдвиге $0b1011_0100 \ll 1$ мы получаем $0b1_0110_1000$ — 9 бит, а регистр вмещает только 8. Старший бит вылез за пределы. Что с ним делать? Если просто выбросить — информация потеряется, и последовательность может выродиться.

Значит нужно «завернуть» старший бит обратно, но не в младший, а в несколько позиций сразу, определённых нашим порождающим. Это и есть применение редукции. Смысл такой: вылетевший бит не теряется, а влияет на несколько битов нового состояния через XOR. Это «перемешивает» информацию гораздо сильнее, чем простой сдвиг. Хотя каждое состояние полностью определяется предыдущим, последовательность битов не выглядит предсказуемой. Чтобы её «угадать», надо знать S_0 и правило перехода.

При правильном порождающем последовательность проходит $2^n - 1$ различных состояний, прежде чем вернуться в начало. На каждом шаге регистр выдает нам один бит - младший бит состояния (положили именно его просто из удобства), и из начального состояния получается длинная последовательность байтов - гамма.

4 Шифрование

Наше сообщение - просто последовательность байтов. Обозначим её $M = [m_0, m_1, m_2, \dots]$. Гамма, которую выдаёт регистр, — тоже последовательность байтов, обозначим её $G = [g_0, g_1, g_2, \dots]$. Шифрование — это XOR каждого байта сообщения с соответствующим байтом гаммы. Получается шифртекст $C = [c_0, c_1, c_2, \dots]$, где

$$c_0 = m_0 \oplus g_0, \quad c_1 = m_1 \oplus g_1, \quad c_2 = m_2 \oplus g_2, \quad \dots$$

Один XOR на каждый байт. На уровне шифрования мы видим только байты.

4.1 Дешифрование

Дешифрование - это та же самая операция, что и шифрование. Тот же XOR, только теперь мы применяем его к шифртексту, а не к открытому сообщению. Получатель знает "секрет" (то же начальное состояние регистра и те же параметры поля), поэтому он может создать у себя точно такой же регистр и получить точно такую же гамму G . Тогда

$$m_0 = c_0 \oplus g_0, \quad m_1 = c_1 \oplus g_1, \quad m_2 = c_2 \oplus g_2, \quad \dots$$

Это работает потому, что XOR - операция, обратная самой себе. Если при шифровании мы наложили гамму, то при дешифровании наложение той же гаммы её снимает. Никаких дополнительных сведений, кроме уже известных получателю, не требуется.

Одна и та же процедура работает и для шифрования, и для дешифрования. В коде это одна функция, которая берёт на вход буфер данных и XOR-ит его с байтами гаммы. Это фундаментальное свойство потокового шифра: секрет - это начальное состояние регистра, и кто им владеет, тот может и зашифровать, и расшифровать.

Чтобы расшифровать сообщение, получателю нужно ровно две вещи: во-первых, знать параметры поля - число n и порождающий многочлен f , во-вторых, знать начальное состояние регистра, то есть тот самый секрет, с которого мы начали генерацию гаммы. Без этих двух частей гамма получается другой, и XOR с ней даёт не исходное сообщение, а бессмысленный набор байтов. Именно поэтому секрет - это ключ: он определяет всю гамму целиком, а значит, и весь процесс шифрования и дешифрования.

Вот как-то так оно и работает примерно. Конец.